

```

        color: Color(0xff2a2e3d), borderRadius: BorderRadius.circular(15)),
child: TextFormField(
  validator: (value){
    // getDataItemName();
    //getDataNick();
    getUniqueItemName();

    if(value!.length < 1){
      return "* Required item name";

    }
    if(docID==true){
      return "* Item already Exists";
      // print("already exists");
    }

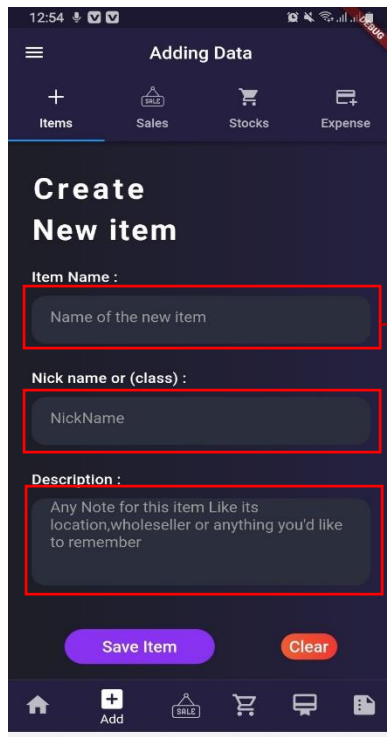
    else{
      return null;
    }
  },
  onSave: (value) => setState(() => itemN=value!),

  controller: _addItemNameController,
  style: TextStyle(color: Colors.grey, fontSize: 17),
  decoration: InputDecoration(
    border: InputBorder.none,
    hintText: "Name of the new item",
    hintStyle: TextStyle(color: Colors.grey, fontSize: 17),
    contentPadding: EdgeInsets.only(
      left: 20,
      right: 20,
    ),
  ),
),
);
}

```

TextFormFields :

As we mention before we made Some self made widgets like textfieldls and buttons to style our Item entry Form and we Have three TextformFields **Item Name** for naming of our items, **Item Nick Name or Class** for specify the category of our items and **Description** to giving some details about our items.



```

Widget nickName() {
  return Container(
    height: 55,
    width: MediaQuery.of(context).size.width,
    decoration: BoxDecoration(
      color: Color(0xff2a2e3d), borderRadius: BorderRadius.circular(15)),
    child: TextFormField(
      validator: (value){
        // getDataItemName();
        //getDataNick();
        getUniqueItemName();

        if(value!.length < 1){
          return "* Required Item NickName";
        }
        if(docID==true){
          return "* Item already exists";
          // print("already exixs");
        }

        else{
          return null;
        }
      },
      onSave: (value) => setState((
        ) =>itemNickN=value!),

      controller: _addItemNickNameControler,

```

```

        style: TextStyle(color: Colors.grey, fontSize: 17),
        decoration: InputDecoration(
          border: InputBorder.none,
          hintText: "NickName",
          hintStyle: TextStyle(color: Colors.grey, fontSize: 17),
          contentPadding: EdgeInsets.only(
            left: 20,
            right: 20,
          ),
        ),
      ),
    ),
  );
}

Widget description() {
  return Container(
    height: 100,
    width: MediaQuery.of(context).size.width,
    decoration: BoxDecoration(
      color: Color(0xff2a2e3d), borderRadius: BorderRadius.circular(15)),
    child: TextFormField(

      controller: _addItemdescriptionControler,
      style: TextStyle(color: Colors.grey, fontSize: 17),
      maxLines: 6,
      decoration: InputDecoration(
        border: InputBorder.none,
        hintText: "Any Note for this item Like its location,wholeseller or
anything you'd like to remember",
        hintStyle: TextStyle(color: Colors.grey, fontSize: 17),
        contentPadding: EdgeInsets.only(
          left: 20,
          right: 20,
        ),
      ),
    ),
  ),
);
}

```

Labels :

As we mention before we used made self textfields we also used the following self made widget For our labels to style for our textfields Labels .

```

Widget label(String label) {
  return Text(
    label,
    style: TextStyle(
      fontSize: 16.5,
      color: Colors.white,
      fontWeight: FontWeight.w600,
      letterSpacing: 0.2),
  );
}

```

Future Function Use Case In this Code :

To get accurate data from users we need compare their entries with our database values. That if there entered values exist or not, in this function we are comparing user entries with Firestore values and we do some validation to make more accurate data entries.

```
Future<dynamic> getUniqueItemName() async {

  try {
    FirebaseFirestore.instance.collection("Users").doc(FirebaseAuth.instance.currentUser!.uid)

      .collection('Item')
      .where("itemName", isEqualTo: _addItemNameController.text)

      .get()

      .then((QuerySnapshot querySnapshot) {

        querySnapshot.docs.forEach((doc) {
          if(doc["itemName"]==_addItemNameController.text
&&doc["itemNickName"]==_addItemNickNameController.text){
            docID = true;
          }
          else{
            docID = false;
          }
        });

      });

  } on Exception catch (e) {
    showSnackBar(context, e.toString());
  }

}
```

```
Widget buttonSave() {
```

```
  Color bgColor;
```

```
  // String value = _addItemNameController.text;
  return InkWell(
```

```
    onTap: ()async {
      setState(() {
        circuler = true;
      });
    },
```

```

        final isValid = formkey.currentState!.validate();

        if(isValid){

            try {
                await FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.i
nstance.currentUser!.uid)
                    .collection("Item").add({
                        "itemName":_addItemNameControler.text,"itemNickName":_addItemNickNameC
ontroler.text,"itemDescription":_addItemdescriptionControler.text,
                        "itemQty":0,"itemCostPer":0.0,"itemTotalCost":0.0,
                    });
                setState(() {
                    circuler = false;
                });
            } on Exception catch (e) {

                showSnackBar(context, e.toString());
                setState(() {
                    circuler = false;
                });
            }
            // Navigator.pop(context);

            formkey.currentState?.save();

            _addItemNameControler.text="";
            _addItemNickNameControler.text = "";
            _addItemdescriptionControler.text="";
            final message = "Item : $itemN $itemNickN Saved Succesfully!";
            final snackBar = SnackBar(
                content: Text(
                    message,
                    style: TextStyle(fontSize: 20),
                ),
                backgroundColor: Colors.blue,
            );
            ScaffoldMessenger.of(context).showSnackBar(snackBar);
            // formkey.currentState?.reset();

        }else{
            final message = "!!! Null Or Duplicate Entries Is Not Acceetable !!! ";
            final snackBar = SnackBar(
                content: Text(
                    message,
                    style: TextStyle(fontSize: 20),
                ),
                backgroundColor: Colors.red,
            );
            ScaffoldMessenger.of(context).showSnackBar(snackBar);
            setState(() {
                circuler = false;
            });
        }
    }
}

```

```

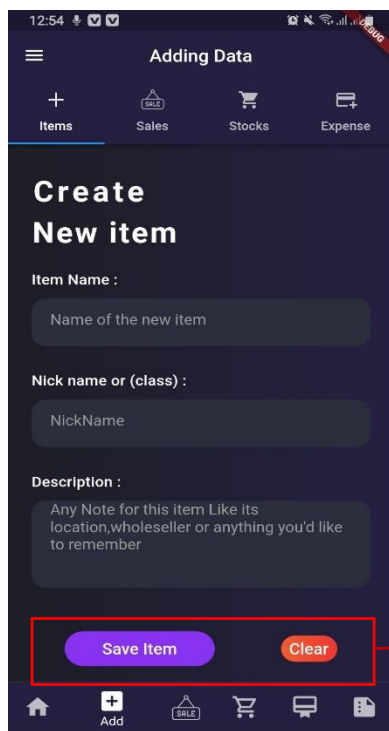
    }

    // Navigator.pop(context);
  },
  child: Container(
    height: 40,
    width: 160,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(20),
      gradient: LinearGradient(colors: [
        Color(0xff8a32f1),
        Color(0xff8a32f1),
      ]),
    ),
    child: Center(child: circular
      ? CircularProgressIndicator()
      : Text("Save Item", style: TextStyle(
        color: Colors.white, fontSize: 18, fontWeight: FontWeight.w600)
    )),
  ),
);
}

```

Save And Clear Buttons :

As you see we made a selfmade widget to style our save button , and also inside the save widget we made interact between IMS App and our firststore database and as we mention before we did some validation on our textformfields to protect null entries or wrong entries . And after the save button we have **Clear Button** if user wants to clear the form



Save And Clear Buttons

```

Widget buttonDelete() {
  return InkWell(
    onTap: (){
      formkey.currentState?.save();
      _addItemNameController.text="";
      _addItemNickNameController.text="";
      _addItemdescriptionController.text="";

      // Navigator.pop(context);
    },
    child: Container(
      height: 30,
      width:60,
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(20),
        gradient: LinearGradient(colors: [
          Color.fromARGB(255, 241, 94, 50),
          Color.fromARGB(255, 241, 56, 50),
        ]),
      ),
      child: Center(child: Text("Clear",style: TextStyle(
        color: Colors.white, fontSize: 18, fontWeight: FontWeight.w600)
      )),),
    ),
  );
}

Future<void> showSnackBar(BuildContext context, String text) async {
  final snackBar = SnackBar(content: Text(text));
  ScaffoldMessenger.of(context).showSnackBar(snackBar);
}
}

```

Stocks And Sales Entry Codes :

The Codes that are used for **Stocks** and **sales Entry** is almost similar so

As a sample we bring stocks entry here.

The structure of Stocks , Sales , item and expenses entries are also same , we use same Container and inside That we use a column to set our labels , textfields and buttons. But the main differences are some functions, Save Buttons and **autotextformfield** that we use in stocks and sales screens.

Coz this we will not bring repeated codes here .

```

Widget StocksitemName() {
  return Container(
    height: 55,
    width: MediaQuery.of(context).size.width,
    decoration: BoxDecoration(
      color: Color(0xff2a2e3d), borderRadius: BorderRadius.circular(15)),

```

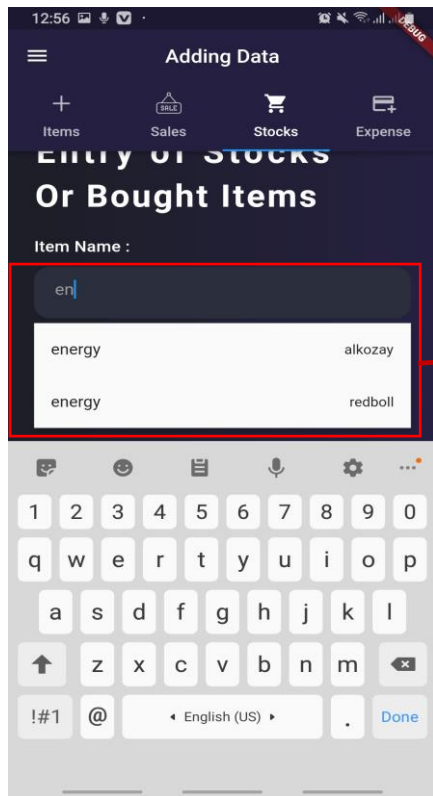
```

child: TypeAheadFormField(
  validator: (value) {
    if (value!.isEmpty) {
      return "* Required item name";
    } else {
      return null;
    }
  },
  textFieldConfiguration: TextFieldConfiguration(
    controller: _suggestionCotroller,
    autofocus: false,
    style: TextStyle(color: Colors.grey, fontSize: 17),
    decoration: InputDecoration(
      border: InputBorder.none,
      hintText: "Name of item you bought",
      hintStyle: TextStyle(color: Colors.grey, fontSize: 17),
      contentPadding: EdgeInsets.only(
        left: 20,
        right: 20,
      ),
    ),
  ),
  suggestionsCallback: (pattern) async {
    return await queryData(pattern);
  },
  itemBuilder: (context, dynamic suggestion) {
    return ListTile(
      title: Text(suggestion["itemName"]),
      trailing: Text(suggestion["itemNickName"]),
    );
  },
  transitionBuilder: (context, suggestionsBox, controller) {
    return suggestionsBox;
  },
  onSuggestionSelected: (dynamic suggestion) {
    _suggestionCotroller.text = suggestion["itemName"];
    //print(suggestion["itemNickName"]);
    //print(suggestion["itemName"]);
    tempItemName = suggestion["itemName"];
    tempItemNickName = suggestion["itemNickName"];
    tempDes = suggestion["itemDescription"];
  },
  onSave: (value) => _suggestionCotroller.text = value!,
),
);
}

```

TypeAheadFormField :

A TypeAhead (autocomplete) widget for Flutter, where you can show suggestions to users as they type, we help users to select valid items that are available in items list.



TypeAheadFormField

```
Widget Stocksquantity() {
  return Container(
    height: 55,
    width: MediaQuery.of(context).size.width,
    decoration: BoxDecoration(
      color: Color(0xff2a2e3d), borderRadius: BorderRadius.circular(15)),
    child: TextFormField(
      validator: (value) {
        if (value!.isEmpty || value == "0" || value == "0.0") {
          return "* Quantity is zero or empty";
        } else {
          return null;
        }
      },
      keyboardType: TextInputType.number,
      inputFormatters: <TextInputFormatter>[
        FilteringTextInputFormatter.digitsOnly
      ],
      controller: _stocksQuantityController,
      style: TextStyle(color: Colors.grey, fontSize: 17),
      decoration: InputDecoration(
        border: InputBorder.none,
        hintText: "No of item you bought",
        hintStyle: TextStyle(color: Colors.grey, fontSize: 17),
        contentPadding: EdgeInsets.only(
          left: 20,
          right: 20,
        ),
      ),
    ),
  ),
);
}
```

Quantity and Pricing Fields :

As you know in every trading forms we will have some pricing and quantity parts that user must Enter just Numbers on that fields, we have **Quantity** , **Per Price** , **total Price** , and **Loan** fields on our stocks and sales screens that validate them to prevent users wrong entries .

Stocks and Sales Save Buttons :

The scenario of saving stocks and sales are very different than saving expenses or and items .

In the following stocks Save widget at first we will have some validation and check if users selects any item to buy or not , after we will update our items data according to stock quantity and pricing deal .

```
Widget StocksbuttonSave() {

  return InkWell(
    onTap: () async {
      setState(() {
        circuler = true;
      });
      final isValid = forStockmkey.currentState!.validate();

      if (isValid) {
        if (tempIteNickmName == "" || tempItemName == "") {
          final message = "!!! Please Select an Item !!! ";
          final snackBar = SnackBar(
            content: Text(
              message,
              style: TextStyle(fontSize: 20),
            ),
            backgroundColor: Colors.red,
          );
          ScaffoldMessenger.of(context).showSnackBar(snackBar);
          setState(() {
            circuler = false;
          });
        } else {

          try {
            await FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
              .collection("Transactions").add({
                "stockName": tempItemName,
                "stockNickName": tempIteNickmName,
                "stockQTY": int.parse(_stocksQuantityController.text),
                "stockCost": double.parse(_stocksPriceCotroller.text),
                "stockTotalCost":
                  double.parse(_stocksTotalPriceCotroller.text),
                "stockLoan":
                  _stocksLoanController.text==""?0.0:double.parse(_stocksLoanController.text),
                "stockDesc": _stocksDescriptionCotroller.text,
                "dateTime": new DateTime.now(),
              }).then((value) async {
```

```

        await
        FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)

            .collection('Item')
            .where("itemName", isEqualTo: tempItemName)
            .where("itemNickName", isEqualTo: tempIteNickmName)
            .get()

            .then((QuerySnapshot querySnapshot) {

                querySnapshot.docs.forEach((doc) {
                    docID = (doc.id);

                });

            });

        await FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
            .collection("Item")
            .doc(docID)
            .update({
                "itemQty": FieldValue.increment(
                    int.parse(_stocksQuantityController.text)),
                "itemCostPer": double.parse(_stocksPriceCotroller.text)
            },
            "itemTotalCost": FieldValue.increment(
                double.parse(_stocksTotalPriceCotroller.text)),
                "itemName":tempItemName,
                "itemNickName":tempIteNickmName,
                "itemDescription":tempDes,
            ));

        });
        forStockmkey.currentState?.save();
        tempItemName = "";
        tempIteNickmName = "";
        tempDes = "";
        docID = "";
        _suggestionCotroller.text="";
        _stocksDescriptionCotroller.text="";
        _stocksLoanController.text="";
        _stocksPriceCotroller.text="";
        _stocksQuantityController.text="";
        _stocksTotalPriceCotroller.text="";
        final message = "Item : $tempItemName $tempIteNickmName Bought
        Successfully!";
        final snackBar = SnackBar(
            content: Text(
                message,
                style: TextStyle(fontSize: 20),
            ),
            backgroundColor: Colors.blue,
        );
        ScaffoldMessenger.of(context).showSnackBar(snackBar);

```

```

        setState(() {
          circuler = false;
        });

        } on Exception catch (e) {
          showSnackBar(context, e.toString());
          setState(() {
            circuler = false;
          });
        }

      }

    } else {
      final message =
        "!!! Null Or Duplicate Entries Is Not Acceetable !!! ";
      final snackBar = SnackBar(
        content: Text(
          message,
          style: TextStyle(fontSize: 20),
        ),
        backgroundColor: Colors.red,
      );
      ScaffoldMessenger.of(context).showSnackBar(snackBar);
      setState(() {
        circuler = false;
      });
    }

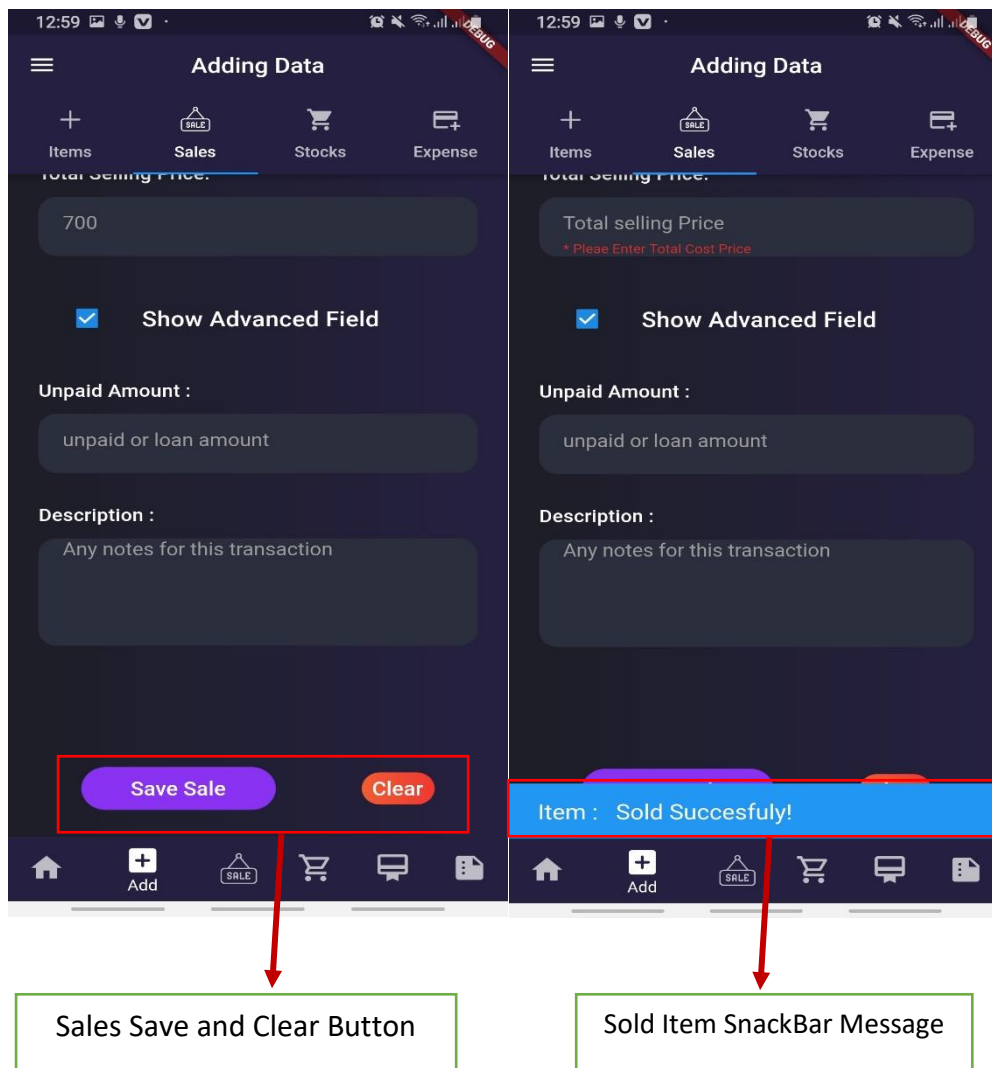
    // Navigator.pop(context);
  },
  child: Container(
    height: 40,
    width: 160,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(20),
      gradient: LinearGradient(colors: [
        Color(0xff8a32f1),
        Color(0xff8a32f1),
      ]),
    ),
  ),
  child: Center(
    child: circuler
      ? CircularProgressIndicator()
      : Text(
        "Save Stock",
        style: TextStyle(
          color: Colors.white, fontSize: 18, fontWeight: FontWeight.w600),
      ),
  ),
),

```

```

    ),
  );
}

```



```

Widget StocksbuttonDelete() {
  return InkWell(
    onTap: () {
      forStockmkey.currentState?.reset();
      tempItemName = "";
      tempIteNickmName = "";
      tempDes = "";
      docID = "";
      _suggestionCotroller.text="";
      _stocksDescriptionCotroller.text="";
      _stocksLoanController.text="";
      _stocksPriceCotroller.text="";
      _stocksQuantityController.text="";
      _stocksTotalPriceCotroller.text="";
    },
    child: Container(
      height: 30,
      width: 60,

```

```

        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(20),
          gradient: LinearGradient(colors: [
            Color.fromARGB(255, 241, 94, 50),
            Color.fromARGB(255, 241, 56, 50),
          ]),
        ),
        child: Center(
          child: Text(
            "Clear",
            style: TextStyle(
              color: Colors.white, fontSize: 18, fontWeight: FontWeight.w600),
          ),
        ),
      ),
    );
  }

  static Future<List> queryData(String suggestion) async =>
    await
    FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
      .collection("Item")
      .where("itemName", isGreaterThanOrEqualTo: suggestion)
      .get()
      .then((snap) {
        return snap.docs;
      });

  static getSuggestion(String suggestion) async =>
    await
    FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
      .collection("Item")
      .where('itemName', isGreaterThanOrEqualTo: suggestion)
      .get()
      .then((snap) {
        return snap.docs;
      });

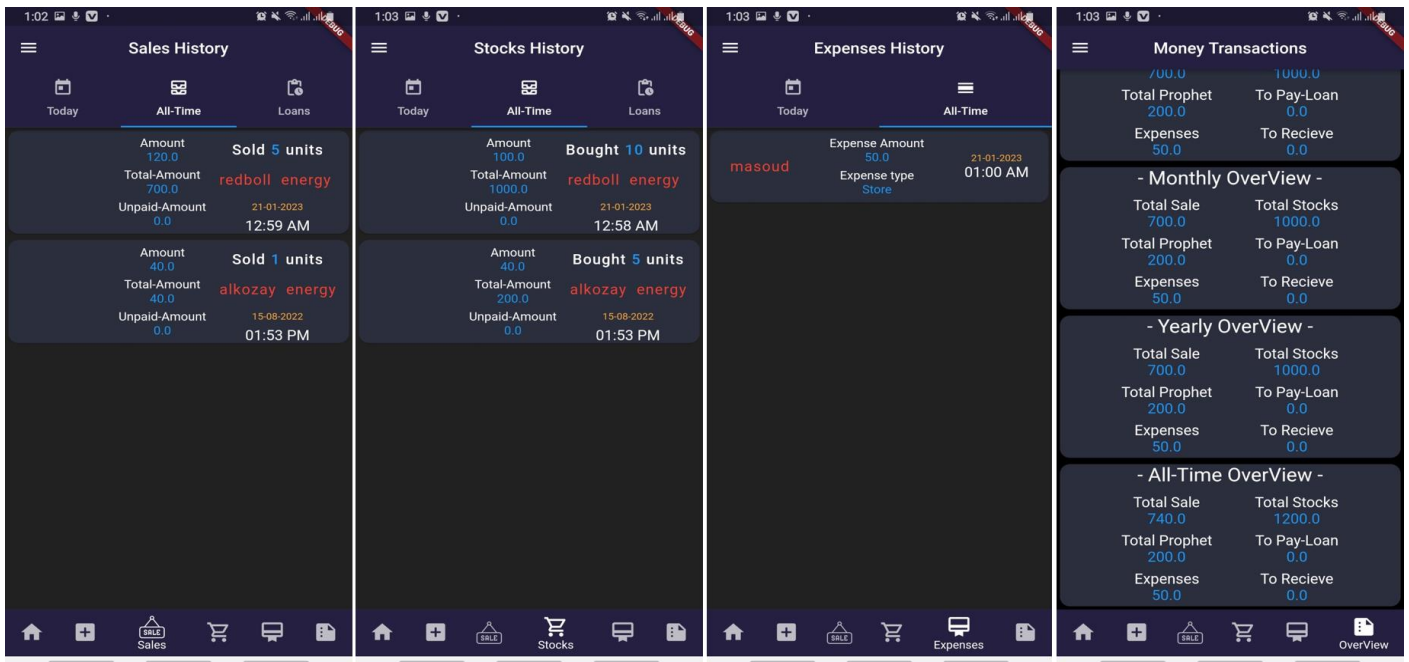
  Future<void> showSnackBar(BuildContext context, String text) async {
    final snackBar = SnackBar(content: Text(text));
    ScaffoldMessenger.of(context).showSnackBar(snackBar);
  }
}

```

4.8 IMS History And Transactions Details

The IMS Application has the most accuracy on retrieving data and on debit or credit. In this section we introduce Sales History , Stocks History, Expenses History And Money Transaction

Screens .



4.8 IMS History And Transactions Screens Design Code Flutter UI

We Have The Same Coding on Sales ,Stocks And Expenses History, and each of them have different Times of retrieving data from Firestor .

For Example we have Sales History Of today And All time And We have a loane Section In each Part And The coding thas used ara almost same.

So we Bring A sample of Sales History .

```
class saleHistory extends StatefulWidget {
  const saleHistory({Key? key}) : super(key: key);

  @override
  State<saleHistory> createState() => _saleHistoryState();
}

class _saleHistoryState extends State<saleHistory> {
  // var dateFormat = new DateFormat();
  AuthClass authClass = AuthClass();
  final Stream<QuerySnapshot> _stream =
    FirebaseFirestore.instance.collection("Iusers").doc(FirebaseAuth.instance.currentUser!.uid)
      .collection("Sales").orderBy("dateTime",descending: true).snapshots();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.black87,

      body: StreamBuilder(
        stream: _stream,
```